

компонентных функций `wait()` или `get()`, приведет к выполнению в синхронном режиме функции `INVOKE(fff, xyz...)` в потоке, который вызвал `wait()` или `get()`.

Из вызова функции `get()` в отношении данного объекта `std::future` будет возвращено либо значение, полученное в результате выполнения функции `INVOKE(fff, xyz...)`, либо исключение, выданное этой функцией.

Если `policy` имеет значение `std::launch::async` | `std::launch::deferred` или же аргумент `policy` опущен, поведение будет таким же, как и при задании политики `std::launch::async` или политики `std::launch::deferred`. Реализация будет выбирать поведение исходя из конкретной обстановки вызова, чтобы задействовать доступную аппаратную конкурентность без превышения лимитов.

Во всех случаях при вызове `std::async` происходит немедленное возвращение управления.

Синхронизация

Завершение вызова функции «происходит до» успешного возвращения из вызова `wait()`, `get()`, `wait_for()` или `wait_until()` в отношении любого экземпляра `std::future` или `std::shared_future`, ссылающегося на то же связанное состояние, что и объект `std::future`, возвращенный из вызова `std::async`. Если `policy` имеет значение `std::launch::async`, завершение потока, в котором произошел вызов функции, также «происходит до» успешного возвращения из этих вызовов.

Выдает

Исключение `std::bad_alloc`, если память под требуемое внутреннее хранилище не может быть выделена, в противном случае выдает исключение `std::future_error`, когда результат не может быть достигнут, или же выдано любое исключение в ходе создания `fff` или `xyz...`.

Г.5. Заголовок `<mutex>`

Заголовок `<mutex>` предоставляет средства обеспечения взаимного исключения: типы мьютексов, типы блокировок и функции, а также механизм, гарантирующий строго однократное выполнение операции.

Содержимое заголовка

```
namespace std
{
    class mutex;
    class recursive_mutex;
    class timed_mutex;
    class recursive_timed_mutex;
    class shared_mutex;
    class shared_timed_mutex;

    struct adopt_lock_t;
    struct defer_lock_t;
    struct try_to_lock_t;

    constexpr adopt_lock_t adopt_lock{};
    constexpr defer_lock_t defer_lock{};
```

```
constexpr try_to_lock_t try_to_lock{};

template<typename LockableType>
class lock_guard;

template<typename LockableType>
class unique_lock;

template<typename LockableType>
class shared_lock;

template<typename ... LockableTypes>
class scoped_lock;

template<typename LockableType1, typename... LockableType2>
void lock(LockableType1& m1, LockableType2& m2...);

template<typename LockableType1, typename... LockableType2>
int try_lock(LockableType1& m1, LockableType2& m2...);

struct once_flag;

template<typename Callable, typename... Args>
void call_once(once_flag& flag, Callable func, Args args...);
}
```

Г.5.1. Класс std::mutex

Класс `std::mutex` предоставляет базовое взаимное исключение и средства синхронизации для потоков, которые могут применяться для защиты совместно используемых данных. До обращения к данным, защищенным мьютексом, этот мьютекс должен быть заблокирован путем вызова функции `lock()` или функции `try_lock()`. Одновременно удерживать мьютекс может только один поток, поэтому если другой поток также пытается заблокировать мьютекс, то он, соответственно, потерпит неудачу (при вызове `try_lock()`) или заблокируется (при вызове `lock()`). Как только поток завершит обращение к совместно используемым данным, он должен вызвать функцию `unlock()` для снятия блокировки и разрешения другим потокам завладеть ею.

Экземпляр `std::mutex` является `Lockable`.

Определение класса

```
class mutex
{
public:
    mutex(mutex const&)=delete;
    mutex& operator=(mutex const&)=delete;

    constexpr mutex() noexcept;
    ~mutex();

    void lock();
    void unlock();
    bool try_lock();
};
```

Конструктор по умолчанию `std::mutex`

Создает объект `std::mutex`.

Объявление

```
constexpr mutex() noexcept;
```

Результат

Создает экземпляр `std::mutex`.

Постусловия

Изначально только что созданный объект `std::mutex` находится в разблокированном состоянии.

Выдает

Ничего не выдает.

Деструктор `std::mutex`

Уничтожает объект `std::mutex`.

Объявление

```
~mutex();
```

Предусловия

Объект `*this` не должен быть заблокирован.

Результат

Уничтожает `*this`.

Выдает

Ничего не выдает.

Компонентная функция `std::mutex::lock`

Получает блокировку объекта `std::mutex` для текущего потока.

Объявление

```
void lock();
```

Предусловия

Вызывающий поток не должен удерживать блокировку объекта `*this`.

Результат

Блокирует текущий поток, пока не будет получена блокировка объекта `*this`.

Постусловия

Объект `*this` заблокирован вызывающим потоком.

Выдает

Исключение типа `std::system_error`, если произошла ошибка.

Компонентная функция `std::mutex::try_lock`

Пытается получить блокировку объекта `std::mutex` для текущего потока.

Объявление

```
bool try_lock();
```

Предусловия

Вызывающий поток не должен удерживать блокировку объекта `*this`.

Результат

Пытается получить блокировку `*this` для вызывающего потока без его блокировки.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком.

Компонентная функция `std::mutex::unlock`

Освобождает блокировку объекта `std::mutex`, удерживаемую текущим потоком.

Объявление

```
void unlock();
```

Предусловия

Вызывающий поток должен удерживать блокировку `*this`.

Результат

Освобождает блокировку `*this`, удерживаемую текущим потоком. Если имеются какие-либо потоки, заблокированные в ожидании получения блокировки `*this`, разблокируется один из них.

Постусловия

Объект `*this` не заблокирован вызывающим потоком.

Выдает

Ничего не выдает.

Г.5.2. Класс `std::recursive_mutex`

Класс `std::recursive_mutex` предоставляет базовое взаимное исключение и средства синхронизации для потоков, которые можно применять для защиты совместно используемых данных. До обращения к данным, защищенным мьютексом, этот мьютекс должен быть заблокирован путем вызова функции `lock()` или функции `try_lock()`. Одновременно удерживать мьютекс может только один поток, поэтому если другой поток также пытается заблокировать `recursive_mutex` мьютекс, то он, соответственно, потерпит неудачу (при вызове `try_lock()`) или заблокируется (при вызове `lock()`). Как только поток завершит обращение к совместно используемым данным, он должен вызвать функцию `unlock()` для снятия блокировки и разрешения другим потокам завладеть ею.

Это рекурсивный мьютекс, поэтому поток, удерживающий блокировку конкретного экземпляра `std::recursive_mutex`, может и дальше вызывать `lock()` или `try_lock()`, чтобы увеличивать количество блокировок. Мьютекс не может быть заблокирован другим потоком, пока поток, получивший блокировку, не вызовет функцию `unlock` столько раз, сколько было успешных вызовов функций `lock()` или `try_lock()`.

Экземпляр `std::recursive_mutex` является `Lockable`.

Определение класса

```
class recursive_mutex
{
public:
    recursive_mutex(recursive_mutex const&)=delete;
    recursive_mutex& operator=(recursive_mutex const&)=delete;
    recursive_mutex() noexcept;
    ~recursive_mutex();

    void lock();
    void unlock();
    bool try_lock() noexcept;
};
```

Конструктор по умолчанию `std::recursive_mutex`

Создает объект `std::recursive_mutex`.

Объявление

```
recursive_mutex() noexcept;
```

Результат

Создает экземпляр `std::recursive_mutex`.

Постусловия

Изначально только что созданный объект `std::recursive_mutex` находится в разблокированном состоянии.

Выдает

Исключение типа `std::system_error`, если не может создать новый экземпляр `std::recursive_mutex`.

Деструктор `std::recursive_mutex`

Уничтожает объект `std::recursive_mutex`.

Объявление

```
~recursive_mutex();
```

Предусловия

Объект `*this` не должен быть заблокирован.

Результат

Уничтожает `*this`.

Выдает

Ничего не выдает.

Компонентная функция `std::recursive_mutex::lock`

Получает блокировку объекта `std::recursive_mutex` для текущего потока.

Объявление

```
void lock();
```

Результат

Блокирует текущий поток, пока не будет получена блокировка `*this`.

Постусловия

Объект `*this` заблокирован вызывающим потоком. Если вызывающий поток уже удерживает блокировку `*this`, значение счетчика блокировок увеличивается на единицу.

Выдает

Исключение типа `std::system_error`, если произошла ошибка.

Компонентная функция `std::recursive_mutex::try_lock`

Пытается получить блокировку объекта `std::recursive_mutex` для текущего потока.

Объявление

```
bool try_lock() noexcept;
```

Результат

Пытается получить блокировку `*this` для вызывающего потока без его блокировки.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Получена новая блокировка `*this` для вызывающего потока, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Если вызывающий поток уже удерживает блокировку `*this`, функция возвращает `true`, а значение счетчика блокировок `*this`, удерживаемых вызывающим потоком, увеличивается на единицу. Если текущий поток еще не удерживает блокировку `*this`, функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком.

Компонентная функция `std::recursive_mutex::unlock`

Освобождает блокировку объекта `std::recursive_mutex`, удерживаемую текущим потоком.

Объявление

```
void unlock();
```

Предусловия

Вызывающий поток должен удерживать блокировку `*this`.

Результат

Освобождает блокировку `*this`, удерживаемую текущим потоком. Если это последняя блокировка `*this`, удерживаемая вызывающим потоком, и есть какие-либо потоки, заблокированные в ожидании получения блокировки `*this`, разблокируется один из них.

Постусловия

Количество блокировок `*this`, удерживаемых вызывающим потоком, уменьшается на единицу.

Выдает

Ничего не выдает.

Г.5.3. Класс `std::timed_mutex`

Класс `std::timed_mutex` предоставляет поддержку блокировок с тайм-аутами в виде надстроек над базовыми взаимными исключениями и средствами синхронизации, предоставляемыми `std::mutex`. До обращения к данным, защищенным мьютексом, этот мьютекс должен быть заблокирован путем вызова одной из функций `lock()`.

`try_lock()`, `try_lock_for()` или `try_lock_until()`. Если блокировка уже удерживается другим потоком, попытка получения блокировки потерпит неудачу (при вызове `try_lock()`), произойдет блокировка потока до появления возможности получения блокировки (при вызове `lock()`) или поток будет заблокирован, пока не появится возможность получить блокировку или не выйдет время на попытку ее получения (при вызове `try_lock_for()` или `try_lock_until()`). Как только блокировка будет получена (при этом неважно, какая именно функция использовалась для ее получения), она должна быть освобождена путем вызова функции `unlock()`, прежде чем мьютекс сможет быть заблокирован другим потоком.

Экземпляр `std::timed_mutex` является `TimedLockable`.

Определение класса

```
class timed_mutex
{
public:
    timed_mutex(timed_mutex const&)=delete;
    timed_mutex& operator=(timed_mutex const&)=delete;

    timed_mutex();
    ~timed_mutex();

    void lock();
    void unlock();
    bool try_lock();

    template<typename Rep,typename Period>
    bool try_lock_for(
        std::chrono::duration<Rep,Period> const& relative_time);

    template<typename Clock,typename Duration>
    bool try_lock_until(
        std::chrono::time_point<Clock,Duration> const& absolute_time);
};
```

Конструктор по умолчанию `std::timed_mutex`

Создает объект `std::timed_mutex`.

Объявление

```
timed_mutex();
```

Результат

Создает экземпляр `std::timed_mutex`.

Постусловия

Изначально только что созданный объект `std::timed_mutex` находится в разблокированном состоянии.

Выдает

Исключение типа `std::system_error`, если не может создать новый экземпляр `std::timed_mutex`.

Деструктор `std::timed_mutex`

Уничтожает объект `std::timed_mutex`.

Объявление

```
~timed_mutex();
```

Предусловия

Объект `*this` не должен быть заблокирован.

Результат

Уничтожает `*this`.

Выдает

Ничего не выдает.

Компонентная функция `std::timed_mutex::lock`

Получает блокировку объекта `std::timed_mutex` для текущего потока.

Объявление

```
void lock();
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Блокирует текущий поток, пока не будет получена блокировка объекта `*this`.

Постусловия

Объект `*this` заблокирован вызывающим потоком.

Выдает

Исключение типа `std::system_error`, если произошла ошибка.

Компонентная функция `std::timed_mutex::try_lock`

Пытается получить блокировку объекта `std::timed_mutex` для текущего потока.

Объявление

```
bool try_lock();
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Пытается получить блокировку `*this` для вызывающего потока без его блокировки.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком.

Компонентная функция `std::timed_mutex::try_lock_for`

Пытается получить блокировку объекта `std::timed_mutex` для текущего потока.

Объявление

```
template<typename Rep, typename Period>  
bool try_lock_for(  
    std::chrono::duration<Rep, Period> const& relative_time);
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Пытается получить блокировку `*this` для вызывающего потока за время, заданное аргументом `relative_time`. Если функция `relative_time.count()` возвращает положительное или отрицательное значение, вызов тут же возвращает управление, как если бы это был вызов функции `try_lock()`. В противном случае вызов блокирует поток, пока не будет получена блокировка или не истечет время, заданное аргументом `relative_time`.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком. Поток может быть заблокирован на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Компонентная функция `std::timed_mutex::try_lock_until`

Пытается получить блокировку объекта `std::timed_mutex` для текущего потока.

Объявление

```
template<typename Clock, typename Duration>
bool try_lock_until(
    std::chrono::time_point<Clock, Duration> const& absolute_time);
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Пытается получить блокировку `*this` для вызывающего потока до времени, заданного аргументом `absolute_time`. Если при входе в функцию `absolute_time <= Clock::now()`, вызов тут же возвращает управление, как если бы это был вызов функции `try_lock()`. В противном случае вызов блокирует поток, пока либо не будет получена блокировка, либо в результате вызова функции `Clock::now()` не будет возвращено время, равное времени, заданному аргументом `absolute_time`, или более позднее.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком. Насчет продолжительности блокировки не дается никаких гарантий, только если функция вернула `false`, вызов `Clock::now()` возвращает время, равное `absolute_time` в тот момент, когда поток стал разблокированным, или более позднее.

Компонентная функция `std::timed_mutex::unlock`

Освобождает блокировку объекта `std::timed_mutex`, удерживаемую текущим потоком.

Объявление

```
void unlock();
```

Предусловия

Вызывающий поток должен удерживать блокировку `*this`.

Результат

Освобождает блокировку объекта **this*, удерживаемую текущим потоком. Если имеются какие-либо потоки, заблокированные в ожидании получения блокировки **this*, разблокируется один из них.

Постусловия

Объект **this* не заблокирован вызывающим потоком.

Выдает

Ничего не выдает.

Г.5.4. Класс `std::recursive_timed_mutex`

Класс `std::recursive_timed_mutex` предоставляет поддержку блокировок с таймаутами в виде надстроек над взаимным исключением и средством синхронизации, предоставляемыми `std::recursive_mutex`. До обращения к данным, защищенным мьютексом, этот мьютекс должен быть заблокирован путем вызова одной из функций `lock()`, `try_lock()`, `try_lock_for()` или `try_lock_until()`. Если блокировка уже удерживается другим потоком, попытка получения блокировки потерпит неудачу (при вызове `try_lock()`), произойдет блокировка потока до появления возможности получения блокировки (при вызове `lock()`) или поток будет заблокирован, пока не появится возможность получить блокировку или не выйдет время на попытку ее получения (при вызове `try_lock_for()` или `try_lock_until()`). Как только блокировка будет получена (при этом неважно, какая именно функция использовалась для ее получения), она должна быть освобождена путем вызова функции `unlock()`, прежде чем мьютекс сможет быть заблокирован другим потоком.

Это рекурсивный мьютекс, поэтому поток, удерживающий блокировку конкретного экземпляра `std::recursive_timed_mutex`, может получать дополнительные блокировки этого экземпляра посредством любой функции получения блокировки. Прежде чем другой поток сможет получить блокировку данного экземпляра, все эти блокировки должны быть освобождены соответствующим вызовом функции `unlock()`.

Экземпляр `std::recursive_timed_mutex` должен быть `TimedLockable`.

Определение класса

```
class recursive_timed_mutex
{
public:
    recursive_timed_mutex(recursive_timed_mutex const&)=delete;
    recursive_timed_mutex& operator=(recursive_timed_mutex const&)=delete;

    recursive_timed_mutex();
    ~recursive_timed_mutex();

    void lock();
    void unlock();
    bool try_lock() noexcept;

    template<typename Rep,typename Period>
```

```

bool try_lock_for(
    std::chrono::duration<Rep,Period> const& relative_time);

template<typename Clock,typename Duration>
bool try_lock_until(
    std::chrono::time_point<Clock,Duration> const& absolute_time);
};

```

Конструктор по умолчанию std::recursive_timed_mutex

Создает объект std::recursive_timed_mutex.

Объявление

```
recursive_timed_mutex();
```

Результат

Создает экземпляр std::recursive_timed_mutex.

Постусловия

Изначально только что созданный объект std::recursive_timed_mutex находится в разблокированном состоянии.

Выдает

Исключение типа std::system_error, если не может создать новый экземпляр std::recursive_timed_mutex.

Деструктор std::recursive_timed_mutex

Уничтожает объект std::recursive_timed_mutex.

Объявление

```
~recursive_timed_mutex();
```

Предусловия

Объект *this не должен быть заблокирован.

Результат

Уничтожает *this.

Выдает

Ничего не выдает.

Компонентная функция std::recursive_timed_mutex::lock

Получает блокировку объекта std::recursive_timed_mutex для текущего потока.

Объявление

```
void lock();
```

Предусловия

Вызывающий поток не должен удерживать блокировку *this.

Результат

Блокирует текущий поток, пока не будет получена блокировка объекта `*this`.

Постусловия

Объект `*this` заблокирован вызывающим потоком. Если вызывающий поток уже удерживает блокировку `*this`, значение счетчика блокировок увеличивается на единицу.

Выдает

Исключение типа `std::system_error`, если произошла ошибка.

Компонентная функция `std::recursive_timed_mutex::try_lock`

Пытается получить блокировку объекта `std::recursive_timed_mutex` для текущего потока.

Объявление

```
bool try_lock() noexcept;
```

Результат

Пытается получить блокировку `*this` для вызывающего потока без его блокировки.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Если вызывающий поток уже удерживает блокировку `*this`, функция возвращает `true`, а значение счетчика блокировок `*this`, удерживаемых вызывающим потоком, увеличивается на единицу. Если текущий поток еще не удерживает блокировку `*this`, функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком.

Компонентная функция `std::recursive_timed_mutex::try_lock_for`

Пытается получить блокировку объекта `std::recursive_timed_mutex` для текущего потока.

Объявление

```
template<typename Rep,typename Period>
bool try_lock_for(
    std::chrono::duration<Rep,Period> const& relative_time);
```

Результат

Пытается получить блокировку **this* для вызывающего потока за время, заданное аргументом *relative_time*. Если функция *relative_time.count()* возвращает положительное или отрицательное значение, вызов тут же возвращает управление, как если бы это был вызов функции *try_lock()*. В противном случае вызов блокирует поток, пока не будет получена блокировка или не истечет время, заданное аргументом *relative_time*.

Возвращает

true, если блокировка была получена для вызывающего потока, и *false* в противном случае.

Постусловия

Объект **this* заблокирован вызывающим потоком, если функция возвратила значение *true*.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Если вызывающий поток уже удерживает блокировку **this*, функция возвращает *true*, а значение счетчика блокировок **this*, удерживаемых вызывающим потоком, увеличивается на единицу. Если текущий поток еще не удерживает блокировку **this*, функция может не получить блокировку (и вернуть значение *false*), даже если блокировка **this* не удерживается никаким другим потоком. Поток может быть заблокирован на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Компонентная функция `std::recursive_timed_mutex::try_lock_until`

Пытается получить блокировку объекта `std::recursive_timed_mutex` для текущего потока.

Объявление

```
template<typename Clock,typename Duration>
bool try_lock_until(
    std::chrono::time_point<Clock,Duration> const& absolute_time);
```

Результат

Пытается получить блокировку **this* для вызывающего потока до времени, заданного аргументом *absolute_time*. Если при входе в функцию *absolute_time <= Clock::now()*, вызов тут же возвращает управление, как если бы это был вызов функции *try_lock()*. В противном случае вызов блокирует поток, пока либо не будет получена блокировка, либо в результате вызова функции *Clock::now()*

не будет возвращено время, равное времени, заданному аргументом `absolute_time`, или более позднее.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Если вызывающий поток уже удерживает блокировку `*this`, функция возвращает `true`, а значение счетчика блокировок `*this`, удерживаемых вызывающим потоком, увеличивается на единицу. Если текущий поток еще не удерживает блокировку `*this`, функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком. Насчет продолжительности блокировки не дается никаких гарантий, только если функция вернула `false`, вызов `Clock::now()` возвращает время, равное `absolute_time` в тот момент, когда поток стал разблокированным, или более позднее.

Компонентная функция `std::recursive_timed_mutex::unlock`

Освобождает блокировку объекта `std::recursive_timed_mutex`, удерживаемую текущим потоком.

Объявление

```
void unlock();
```

Предусловия

Вызывающий поток должен удерживать блокировку `*this`.

Результат

Освобождает блокировку объекта `*this`, удерживаемую текущим потоком. Если это последняя блокировка `*this`, удерживаемая вызывающим потоком, и есть какие-либо потоки, заблокированные в ожидании получения блокировки `*this`, разблокируется один из них.

Постусловия

Количество блокировок `*this`, удерживаемых вызывающим потоком, уменьшается на единицу.

Выдает

Ничего не выдает.

Г.5.5. Класс `std::shared_mutex`

Класс `std::shared_mutex` предоставляет взаимное исключение и средства синхронизации для потоков, которые можно применять для защиты совместно используемых данных, которые часто считываются и редко изменяются. Он позволяет одному потоку удерживать исключительную блокировку или одному или нескольким потокам удерживать совместно используемую блокировку. До изменения данных, защищенных мьютексом, этот мьютекс должен получить исключительную блокировку путем вызова функции `lock()` или функции `try_lock()`. Одновременно исключительная блокировка может удерживаться только одним потоком, поэтому если другой поток также попытается заблокировать мьютекс, то он, соответственно, потерпит неудачу (при вызове `try_lock()`) или заблокируется (при вызове `lock()`). Как только поток завершит внесение изменений в совместно используемые данные, он должен вызвать функцию `unlock()` для снятия блокировки и разрешения другим потокам завладеть ею. Потоки, которым нужно лишь прочитать защищаемые данные, могут получить совместно используемую блокировку путем вызова функции `lock_shared()` или функции `try_lock_shared()`. Одновременно удерживать совместно используемую блокировку могут сразу несколько потоков, поэтому, если один поток удерживает такую блокировку, то другой поток может получить такую же блокировку. Если поток попытается получить исключительную блокировку, ему придется подождать. Как только поток, получивший совместно используемую блокировку, завершит обращение к защищаемым данным, он должен вызвать функцию `unlock_shared()`, чтобы освободить совместно используемую блокировку.

Экземпляр `std::shared_mutex` является `Lockable`.

Определение класса

```
class shared_mutex
{
public:
    shared_mutex(shared_mutex const&)=delete;
    shared_mutex& operator=(shared_mutex const&)=delete;

    shared_mutex() noexcept;
    ~shared_mutex();

    void lock();
    void unlock();
    bool try_lock();

    void lock_shared();
    void unlock_shared();
    bool try_lock_shared();
};
```

Конструктор по умолчанию `std::shared_mutex`

Создает объект `std::shared_mutex`.

Объявление

```
shared_mutex() noexcept;
```

Результат

Создает экземпляр `std::shared_mutex`.

Постусловия

Изначально только что созданный объект `std::shared_mutex` находится в разблокированном состоянии.

Выдает

Ничего не выдает.

Деструктор `std::shared_mutex`

Уничтожает объект `std::shared_mutex`.

Объявление

```
~shared_mutex();
```

Предусловия

Объект `*this` не должен быть заблокирован.

Результат

Уничтожает `*this`.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_mutex::lock`

Получает исключительную блокировку объекта `std::shared_mutex` для текущего потока.

Объявление

```
void lock();
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Блокирует текущий поток до появления возможности получения исключительной блокировки объекта `*this`.

Постусловия

Объект `*this` заблокирован вызывающим потоком с применением исключительной блокировки.

Выдает

Исключение типа `std::system_error`, если произошла ошибка.

Компонентная функция `std::shared_mutex::try_lock`

Пытается получить исключительную блокировку объекта `std::shared_mutex` для текущего потока.

Объявление

```
bool try_lock();
```

Предусловия

Вызывающий поток не должен удерживать блокировку **this*.

Результат

Пытается получить исключительную блокировку **this* для вызывающего потока без его блокировки.

Возвращает

true, если блокировка была получена для вызывающего потока, и *false* в противном случае.

Постусловия

Объект **this* заблокирован вызывающим потоком с применением исключительной блокировки, если функция возвратила *true*.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение *false*), даже если блокировка **this* не удерживается никаким другим потоком.

Компонентная функция `std::shared_mutex::unlock`

Освобождает исключительную блокировку объекта `std::shared_mutex`, удерживаемую текущим потоком.

Объявление

```
void unlock();
```

Предусловия

Вызывающий поток должен удерживать блокировку **this*.

Результат

Освобождает исключительную блокировку объекта **this*, удерживаемую текущим потоком. Если какие-либо потоки заблокированы в ожидании получения блокировки **this*, разблокирует один поток, ожидающий получения исключительно блокировки или несколько потоков, ожидающих получения совместно используемой блокировки.

Постусловия

Объект **this* не заблокирован вызывающим потоком.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_mutex::lock_shared`

Получает совместно используемую блокировку объекта `std::shared_mutex` для текущего потока.

Объявление

```
void lock_shared();
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Блокирует текущий поток, пока не будет получена совместно используемая блокировка `*this`.

Постусловия

Объект `*this` заблокирован вызывающим потоком с применением совместно используемой блокировки.

Выдает

Исключение типа `std::system_error`, если произошла ошибка.

Компонентная функция `std::shared_mutex::try_lock_shared`

Пытается получить совместно используемую блокировку объекта `std::shared_mutex` для текущего потока.

Объявление

```
bool try_lock_shared();
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Пытается получить совместно используемую блокировку объекта `*this` для вызывающего потока без его блокировки.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком с применением совместно используемой блокировки, если функция возвращает `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком.

Компонентная функция `std::shared_mutex::unlock_shared`

Освобождает совместно используемую блокировку объекта `std::shared_mutex`, удерживаемую текущим потоком.

Объявление

```
void unlock_shared();
```

Предусловия

Вызывающий поток должен удерживать блокировку `*this`.

Результат

Освобождает совместно используемую блокировку объекта `*this`, удерживаемую текущим потоком. Если это последняя совместно используемая блокировка `*this` и есть какие-либо потоки, заблокированные в ожидании получения блокировки `*this`, разблокируется один поток, ожидающий эксклюзивной блокировки, или несколько потоков, ожидающих совместно используемой блокировки.

Постусловия

Объект `*this` не заблокирован вызывающим потоком.

Выдает

Ничего не выдает.

Г.5.6. Класс `std::shared_timed_mutex`

Класс `std::shared_mutex` предоставляет поддержку блокировок с тайм-аутами в виде надстроек над взаимным исключением и средством синхронизации для потоков, которые можно применять для защиты совместно используемых данных, которые часто считываются и редко изменяются. Он позволяет удерживать одному потоку исключительную блокировку или одному или нескольким потокам — совместно используемую блокировку. До изменения данных, защищенных мьютексом, этот мьютекс должен получить исключительную блокировку путем вызова функции `lock()` или функции `try_lock()`. Одновременно исключительная блокировка может удерживаться только одним потоком, поэтому если другой поток также попытается заблокировать мьютекс, то он, соответственно, потерпит неудачу (при вызове `try_lock()`) или заблокируется (при вызове `lock()`). Как только поток завершит внесение изменений в совместно используемые данные, он должен вызвать функцию `unlock()` для снятия блокировки и разрешения другим потокам завладеть ею. Потоки, которым нужно только прочитать защищаемые данные, могут получить совместно используемую блокировку путем вызова функции `lock_shared()` или `try_lock_shared()`. Одновременно удерживать совместно используемую блокировку могут сразу несколько потоков, поэтому если один поток удерживает такую блокировку, то другой поток может получить такую же блокировку. Если поток пытается получить исключительную блокировку, ему придется подождать. Как только поток, получивший совместно используемую блокировку, завершит обращение к защищаемым данным, он должен вызвать функцию `unlock_shared()`, чтобы освободить совместно используемую блокировку.

Экземпляр `std::shared_timed_mutex` должен быть `lockable`.

Определение класса

```
class shared_timed_mutex
{
public:
    shared_timed_mutex(shared_timed_mutex const&)=delete;
    shared_timed_mutex& operator=(shared_timed_mutex const&)=delete;

    shared_timed_mutex() noexcept;
    ~shared_timed_mutex();

    void lock();
    void unlock();
    bool try_lock();

    template<typename Rep,typename Period>
    bool try_lock_for(
        std::chrono::duration<Rep,Period> const& relative_time);

    template<typename Clock,typename Duration>
    bool try_lock_until(
        std::chrono::time_point<Clock,Duration> const& absolute_time);

    void lock_shared();
    void unlock_shared();
    bool try_lock_shared();

    template<typename Rep,typename Period>
    bool try_lock_shared_for(
        std::chrono::duration<Rep,Period> const& relative_time);

    template<typename Clock,typename Duration>
    bool try_lock_shared_until(
        std::chrono::time_point<Clock,Duration> const& absolute_time);
};
```

Конструктор по умолчанию `std::shared_timed_mutex`

Создает объект `std::shared_timed_mutex`.

Объявление

```
shared_timed_mutex() noexcept;
```

Результат

Создает экземпляр `std::shared_timed_mutex`.

Постусловия

Изначально только что созданный объект `std::shared_timed_mutex` находится в разблокированном состоянии.

Выдает

Ничего не выдает.

Деструктор `std::shared_timed_mutex`

Уничтожает объект `std::shared_timed_mutex`.

Объявление

```
~shared_timed_mutex();
```

Предусловия

Объект `*this` не должен быть заблокирован.

Результат

Уничтожает `*this`.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_timed_mutex::lock`

Получает исключительную блокировку объекта `std::shared_timed_mutex` для текущего потока.

Объявление

```
void lock();
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Блокирует текущий поток, пока появится возможность получения исключительной блокировки `*this`.

Постусловия

Объект `*this` заблокирован вызывающим потоком с применением исключительной блокировки.

Выдает

Исключение типа `std::system_error`, если произошла ошибка.

Компонентная функция `std::shared_timed_mutex::try_lock`

Пытается получить исключительную блокировку объекта `std::shared_timed_mutex` для текущего потока.

Объявление

```
bool try_lock();
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Пытается получить исключительную блокировку объекта `*this` для вызывающего потока без его блокировки.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком с применением исключительной блокировки, если функция возвращает `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком.

Компонентная функция `std::shared_timed_mutex::try_lock_for`

Пытается получить исключительную блокировку объекта `std::shared_timed_mutex` для текущего потока.

Объявление

```
template<typename Rep, typename Period>
bool try_lock_for(
    std::chrono::duration<Rep, Period> const& relative_time);
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Пытается получить исключительную блокировку `*this` для вызывающего потока за время, заданное аргументом `relative_time`. Если функция `relative_time.count()` возвращает ноль или отрицательное значение, вызов тут же возвращает управление, как если бы это был вызов функции `try_lock()`. В противном случае вызов блокирует поток, пока не будет получена блокировка или не истечет время, заданное аргументом `relative_time`.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком. Поток может быть заблокирован на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Компонентная функция `std::shared_timed_mutex::try_lock_until`

Пытается получить исключительную блокировку объекта `std::shared_timed_mutex` для текущего потока.

Объявление

```
template<typename Clock, typename Duration>
bool try_lock_until(
    std::chrono::time_point<Clock, Duration> const& absolute_time);
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Пытается получить исключительную блокировку `*this` для вызывающего потока до времени, заданного аргументом `absolute_time`. Если при входе в функцию `absolute_time <= Clock::now()`, вызов тут же возвращает управление, как если бы это был вызов функции `try_lock()`. В противном случае вызов блокирует поток, пока либо не будет получена блокировка, либо в результате вызова функции `Clock::now()` не будет возвращено время, равное времени, заданному аргументом `absolute_time`, или более позднее.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком. Насчет продолжительности блокировки не дается никаких гарантий, только если функция вернула `false`, вызов `Clock::now()` возвращает время, равное `absolute_time` в тот момент, когда поток стал разблокированным, или более позднее.

Компонентная функция `std::shared_timed_mutex::unlock`

Освобождает исключительную блокировку объекта `std::shared_timed_mutex`, удерживаемую текущим потоком.

Объявление

```
void unlock();
```

Предусловия

Вызывающий поток должен удерживать блокировку `*this`.

Результат

Освобождает исключительную блокировку объекта `*this`, удерживаемую текущим потоком. Если имеются какие-либо потоки, заблокированные в ожидании получения блокировки `*this`, разблокируется один поток, ожидающий получения исключительной блокировки, или несколько потоков, ожидающих получения совместно используемой блокировки.

Постусловия

Объект `*this` не заблокирован вызывающим потоком.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_timed_mutex::lock_shared`

Получает совместно используемую блокировку объекта `std::shared_timed_mutex` для текущего потока.

Объявление

```
void lock_shared();
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Блокирует текущий поток, пока не появится возможность получения совместно используемой блокировки `*this`.

Постусловия

Объект `*this` заблокирован вызывающим потоком с применением совместно используемой блокировки.

Выдает

Исключение типа `std::system_error`, если произошла ошибка.

Компонентная функция `std::shared_timed_mutex::try_lock_shared`

Пытается получить совместно используемую блокировку объекта `std::shared_timed_mutex` для текущего потока.

Объявление

```
bool try_lock_shared();
```

Предусловия

Вызывающий поток не должен удерживать блокировку **this*.

Результат

Пытается получить совместно используемую блокировку объекта **this* для вызывающего потока без его блокировки.

Возвращает

true, если блокировка была получена для вызывающего потока, и *false* в противном случае.

Постусловия

Объект **this* заблокирован вызывающим потоком с применением совместно используемой блокировки, если функция возвращает *true*.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение *false*), даже если блокировка **this* не удерживается никаким другим потоком.

Компонентная**функция `std::shared_timed_mutex::try_lock_shared_for`**

Пытается получить совместно используемую блокировку объекта `std::shared_timed_mutex` для текущего потока.

Объявление

```
template<typename Rep,typename Period>
bool try_lock_for(
    std::chrono::duration<Rep,Period> const& relative_time);
```

Предусловия

Вызывающий поток не должен удерживать блокировку **this*.

Результат

Пытается получить совместно используемую блокировку **this* для вызывающего потока за время, заданное аргументом *relative_time*. Если функция *relative_time.count()* возвращает ноль или отрицательное значение, вызов тут же возвращает управление, как если бы это был вызов функции *try_lock()*. В противном случае вызов блокирует поток, пока не будет получена блокировка или не истечет время, заданное аргументом *relative_time*.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение `false`), даже если блокировка `*this` не удерживается никаким другим потоком. Поток может быть заблокирован на время дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Компонентная функция `std::shared_timed_mutex::try_lock_until`

Пытается получить совместно используемую блокировку объекта `std::shared_timed_mutex` для текущего потока.

Объявление

```
template<typename Clock, typename Duration>
bool try_lock_until(
    std::chrono::time_point<Clock, Duration> const& absolute_time);
```

Предусловия

Вызывающий поток не должен удерживать блокировку `*this`.

Результат

Пытается получить совместно используемую блокировку `*this` для вызывающего потока до времени, заданного аргументом `absolute_time`. Если при входе в функцию `absolute_time <= Clock::now()`, вызов тут же возвращает управление, как если бы это был вызов функции `try_lock()`. В противном случае вызов блокирует поток, пока либо не будет получена блокировка, либо в результате вызова функции `Clock::now()` не будет возвращено время, равное времени, заданному аргументом `absolute_time`, или более позднее.

Возвращает

`true`, если блокировка была получена для вызывающего потока, и `false` в противном случае.

Постусловия

Объект `*this` заблокирован вызывающим потоком, если функция возвратила значение `true`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Функция может не получить блокировку (и вернуть значение false), даже если блокировка `*this` не удерживается никаким другим потоком. Насчет продолжительности блокировки не дается никаких гарантий, только если функция вернула false, вызов `Clock::now()` возвращает время, равное `absolute_time` в тот момент, когда поток стал разблокированным, или более позднее.

Компонентная функция `std::shared_timed_mutex::unlock_shared`

Освобождает совместно используемую блокировку объекта `std::shared_timed_mutex`, удерживаемую текущим потоком.

Объявление

```
void unlock_shared();
```

Предусловия

Вызывающий поток должен удерживать блокировку `*this`.

Результат

Освобождает совместно используемую блокировку объекта `*this`, удерживаемую текущим потоком. Если это последняя совместно используемая блокировка `*this` и есть какие-либо потоки, заблокированные в ожидании получения блокировки `*this`, разблокируется один поток, ожидающий эксклюзивной блокировки, или несколько потоков, ожидающих совместно используемой блокировки.

Постусловия

Объект `*this` не заблокирован вызывающим потоком.

Выдает

Ничего не выдает.

Г.5.7. Шаблон класса `std::lock_guard`

Шаблон класса `std::lock_guard` предоставляет простую оболочку для принадлежности блокировки. Тип блокируемого мьютекса задается параметром шаблона `Mutex` и должен быть `Lockable`. Указанный мьютекс блокируется в конструкторе и разблокируется в деструкторе. Тем самым предоставляется простое средство блокировки мьютекса для блока кода и предоставления гарантии, что мьютекс будет разблокирован при выходе управления из блока независимо от того, произойдет ли это по достижении конца блока за счет неиспользования инструкции управляющего потока, например `break` или `return`, или за счет выдачи исключения.

Экземпляры `std::lock_guard` являются `MoveConstructible`, `CopyConstructible` или `CopyAssignable`.

Определение класса

```
template <class Mutex>
class lock_guard
{
public:
    typedef Mutex mutex_type;

    explicit lock_guard(mutex_type& m);
    lock_guard(mutex_type& m, adopt_lock_t);
    ~lock_guard();

    lock_guard(lock_guard const& ) = delete;
    lock_guard& operator=(lock_guard const& ) = delete;
};
```

Блокирующий конструктор std::lock_guard

Создает экземпляр `std::lock_guard`, который блокирует предоставленный мьютекс.

Объявление

```
explicit lock_guard(mutex_type& m);
```

Результат

Создает экземпляр `std::lock_guard`, который ссылается на предоставленный мьютекс. Вызывает `m.lock()`.

Выдает

Любое исключение, выданное функцией `m.lock()`.

Постусловия

Объекту `*this` принадлежит блокировка `m`.

Принимающий блокировку конструктор std::lock_guard

Создает экземпляр `std::lock_guard`, который владеет предоставленным мьютексом.

Объявление

```
lock_guard(mutex_type& m, std::adopt_lock_t);
```

Предусловия

Вызывающий поток должен владеть блокировкой `m`.

Результат

Создает экземпляр `std::lock_guard`, ссылающийся на предоставленный мьютекс и получающий владение блокировкой `m`, удерживаемой вызывающим потоком.

Выдает

Ничего не выдает.

Постусловия

Объект `*this` владеет блокировкой `m`, удерживаемой вызывающим потоком.

Деструктор `std::lock_guard`

Уничтожает экземпляр `std::lock_guard` и разблокирует соответствующий мьютекс.

Объявление

```
~lock_guard();
```

Результат

Вызывает функцию `m.unlock()` для экземпляра мьютекса, `m`, представленного при создании объекта `*this`.

Выдает

Ничего не выдает.

Г.5.8. Шаблон класса `std::scoped_lock`

Шаблон класса `std::scoped_lock` предоставляет простую оболочку принадлежности сразу для нескольких мьютексов. Тип блокируемого мьютекса задается пакетом параметров шаблона `Mutexes`, где каждый параметр должен быть `Lockable`. Указанные мьютексы блокируются в конструкторе и разблокируются в деструкторе. Тем самым предоставляется простое средство блокировки набора мьютексов для блока кода и предоставления гарантии, что мьютексы будут разблокированы при выходе управления из блока независимо от того, произошел он по достижении конца блока за счет неиспользования инструкции управляющего потока, например `break` или `return`, или за счет выдачи исключения.

Экземпляры `std::scoped_lock` не являются `MoveConstructible`, `CopyConstructible` или `CopyAssignable`.

Определение класса

```
template <class ... Mutexes>
class scoped_lock
{
public:
    explicit scoped_lock(Mutexes& ... m);
    scoped_lock(Mutexes& ... m, adopt_lock_t);
    ~scoped_lock();

    scoped_lock(scoped_lock const& ) = delete;
    scoped_lock& operator=(scoped_lock const& ) = delete;
};
```

Блокирующий конструктор `std::scoped_lock`

Создает экземпляр `std::scoped_lock`, который блокирует предоставленные мьютексы.

Объявление

```
explicit scoped_lock(Mutexes& ... m);
```

Результат

Создает экземпляр `std::scoped_lock`, ссылающийся на предоставленные мьютексы. Использует комбинацию вызовов `m.lock()`, `m.try_lock()` и `m.unlock()` к каждому из мьютексов во избежание взаимной блокировки, применяя такой же алгоритм, что и свободная функция `std::lock()`.

Выдает

Любые исключения, выданные вызовами `m.lock()` и `m.try_lock()`.

Постусловия

Объект `*this` владеет блокировкой предоставленных мьютексов.

Принимающий блокировку конструктор `std::scoped_lock`

Создает экземпляр `std::scoped_lock`, владеющий блокировкой предоставленных мьютексов; они уже должны быть заблокированы вызывающим потоком.

Объявление

```
scoped_lock(Mutexes& ... m, std::adopt_lock_t);
```

Предусловия

Вызывающий поток должен владеть блокировкой мьютексов в `m`.

Результат

Создает экземпляр `std::scoped_lock`, ссылающийся на предоставленные мьютексы и получающий владение блокировкой мьютексов в `m`, удерживаемой вызывающим потоком.

Выдает

Ничего не выдает.

Постусловия

Объект `*this` владеет блокировкой предоставленных мьютексов, удерживаемой вызывающим потоком.

Деструктор `std::scoped_lock`

Уничтожает экземпляр `std::scoped_lock` и разблокирует соответствующие мьютексы.

Объявление

```
~scoped_lock();
```

Результат

Вызывает `m.unlock()` для каждого экземпляра мьютекса `m`, предоставленного при создании объекта `*this`.

Выдает

Ничего не выдает.

Г.5.9. Шаблон класса `std::unique_lock`

Шаблон класса `std::unique_lock` предоставляет оболочку принадлежности блокировки более общего плана, чем `std::lock_guard`.

Тип блокируемого мьютекса задается параметром шаблона `Mutex` и должен быть `BasicLockable`. Как правило, указанный мьютекс блокируется в конструкторе и разблокируется в деструкторе, но предоставляются также дополнительные конструкторы и компонентные функции, допускающие другие возможности. Тем самым дается средство блокировки мьютекса для блока кода и предоставления гарантии, что мьютекс будет разблокирован при выходе управления из блока независимо от того, произошел он по достижении конца блока за счет использования инструкции управляющего потока, например `break` или `return`, или за счет выдачи исключения. Экземпляры `std::lock_guard` не являются `MoveConstructible`, `CopyConstructible` или `CopyAssignable`. Функции ожидания в классе `std::condition_variable` требуют экземпляра `std::unique_lock<std::mutex>`, и все экземпляры `std::unique_lock` подходят для использования с параметром `Lockable` для функций ожидания `std::condition_variable_any`.

Если предоставляемый тип `Mutex` является `Lockable`, то им также является объект `std::unique_lock<Mutex>`. Если в дополнение к этому предоставляемый тип `Mutex` является `TimedLockable`, то им также является и объект `std::unique_lock<Mutex>`.

Экземпляры `std::unique_lock` являются `MoveConstructible` и `MoveAssignable`, но не `CopyConstructible` или `CopyAssignable`.

Определение класса

```
template <class Mutex>
class unique_lock
{
public:
    typedef Mutex mutex_type;

    unique_lock() noexcept;
    explicit unique_lock(mutex_type& m);
    unique_lock(mutex_type& m, adopt_lock_t);
    unique_lock(mutex_type& m, defer_lock_t) noexcept;
    unique_lock(mutex_type& m, try_to_lock_t);

    template<typename Clock, typename Duration>
    unique_lock(
        mutex_type& m,
        std::chrono::time_point<Clock, Duration> const& absolute_time);

    template<typename Rep, typename Period>
    unique_lock(
        mutex_type& m,
        std::chrono::duration<Rep, Period> const& relative_time);

    ~unique_lock();

    unique_lock(unique_lock const& ) = delete;
```

```

unique_lock& operator=(unique_lock const& ) = delete;

unique_lock(unique_lock&& );
unique_lock& operator=(unique_lock&& );

void swap(unique_lock& other) noexcept;

void lock();
bool try_lock();
template<typename Rep, typename Period>
bool try_lock_for(
    std::chrono::duration<Rep,Period> const& relative_time);
template<typename Clock, typename Duration>
bool try_lock_until(
    std::chrono::time_point<Clock,Duration> const& absolute_time);
void unlock();

explicit operator bool() const noexcept;
bool owns_lock() const noexcept;
Mutex* mutex() const noexcept;
Mutex* release() noexcept;
};

```

Конструктор по умолчанию std::unique_lock

Создает экземпляр std::unique_lock без связанного с ним мьютекса.

Объявление

```
unique_lock() noexcept;
```

Результат

Создает экземпляр std::unique_lock, у которого нет связанного с ним мьютекса.

Постусловия

```
this->mutex()==NULL, this->owns_lock()==false
```

Блокирующий конструктор std::unique_lock

Создает экземпляр std::unique_lock, блокирующий предоставленный мьютекс.

Объявление

```
explicit unique_lock(mutex_type& m);
```

Результат

Создает экземпляр std::unique_lock, ссылающийся на предоставленный мьютекс. Вызывает m.lock().

Выдает

Любые исключения, выданные m.lock().

Постусловия

```
this->owns_lock()==true, this->mutex()==&m
```

Принимающий блокировку конструктор `std::unique_lock`

Создаст экземпляр `std::unique_lock`, владеющий блокировкой предоставленного мьютекса.

Объявление

```
unique_lock(mutex_type& m, std::adopt_lock_t);
```

Предусловия

Вызывающий поток должен владеть блокировкой `m`.

Результат

Создаст экземпляр `std::unique_lock`, ссылающийся на предоставленный мьютекс и получающий владение блокировкой `m`, удерживаемой вызывающим потоком.

Выдает

Ничего не выдает.

Постусловия

```
this->owns_lock()==true, this->mutex()==&m
```

Конструктор отложенной блокировки `std::unique_lock`

Создаст экземпляр `std::unique_lock`, не владеющий блокировкой предоставленного мьютекса.

Объявление

```
unique_lock(mutex_type& m, std::defer_lock_t) noexcept;
```

Результат

Создаст экземпляр `std::unique_lock`, ссылающийся на предоставленный мьютекс.

Выдает

Ничего не выдает.

Постусловия

```
this->owns_lock()==false, this->mutex()==&m
```

Конструктор попытки блокирования `std::unique_lock`

Создаст экземпляр `std::unique_lock`, связанный с предоставленным мьютексом и предпринимающий попытку заблокировать этот мьютекс.

Объявление

```
unique_lock(mutex_type& m, std::try_to_lock_t);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::unique_lock`, должен быть `Lockable`.

Результат

Создает экземпляр `std::unique_lock`, ссылающийся на предоставленный мьютекс.

Вызывает `m.try_lock()`.

Выдает

Ничего не выдает.

Постусловия

Функция `this->owns_lock()` возвращает результат вызова `m.try_lock()`, `this->mutex()==&m`.

Конструктор попытки блокирования с тайм-аутом продолжительности `std::unique_lock`

Создает экземпляр `std::unique_lock`, связанный с предоставленным мьютексом, и пытается получить блокировку этого мьютекса.

Объявление

```
template<typename Rep, typename Period>
unique_lock(
    mutex_type& m,
    std::chrono::duration<Rep, Period> const& relative_time);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::unique_lock`, должен быть `TimedLockable`.

Результат

Создает экземпляр `std::unique_lock`, ссылающийся на предоставленный мьютекс. Вызывает `m.try_lock_for(relative_time)`.

Выдает

Ничего не выдает.

Постусловия

Функция `this->owns_lock()` возвращает результат вызова `m.try_lock_for()`, `this->mutex()==&m`.

Конструктор попытки блокирования с тайм-аутом по моменту времени `time_point` `std::unique_lock`

Создает экземпляр `std::unique_lock`, связанный с предоставленным мьютексом, и пытается получить блокировку этого мьютекса.

Объявление

```
template<typename Clock,typename Duration>
unique_lock(
    mutex_type& m,
    std::chrono::time_point<Clock,Duration> const& absolute_time);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::unique_lock`, должен быть `TimedLockable`.

Результат

Создает экземпляр `std::unique_lock`, ссылающийся на предоставленный мьютекс. Вызывает `m.try_lock_until(absolute_time)`.

Выдает

Ничего не выдает.

Постусловия

Функция `this->owns_lock()` возвращает результат вызова `m.try_lock_until()`, `this->mutex()==&m`.

Перемещающий конструктор `std::unique_lock`

Перемещает принадлежность блокировки из одного объекта `std::unique_lock` в только что созданный объект `std::unique_lock`.

Объявление

```
unique_lock(unique_lock&& other) noexcept;
```

Результат

Создает экземпляр `std::unique_lock`. Если `other` владел блокировкой мьютекса до вызова конструктора, то теперь этой блокировкой владеет только что созданный объект `std::unique_lock`.

Постусловия

Для только что созданного объекта `std::unique_lock`, `x`, результат вычисления `x.mutex()` равен значению `other.mutex()` до вызова конструктора, а результат вычисления `x.owns_lock()` равен значению `other.owns_lock()` до вызова конструктора. `other.mutex()==NULL`, `other.owns_lock()==false`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Объекты `std::unique_lock` не являются `CopyConstructible`, поэтому копирующий конструктор отсутствует и имеется только этот перемещающий конструктор.

Перемещающий оператор присваивания `std::unique_lock`

Перемещает принадлежность блокировки из одного объекта `std::unique_lock` в другой объект `std::unique_lock`.

Объявление

```
unique_lock& operator=(unique_lock&& other) noexcept;
```

Результат

Если `this->owns_lock()` возвращает `true` до вызова, вызывает `this->unlock()`. Если `other` владеет совместно используемой блокировкой мьютекса до присваивания, то теперь этой блокировкой владеет `*this`.

Постусловия

Значение `this->mutex()` равно значению `other.mutex()` до присваивания, а значение `this->owns_lock()` равно значению `other.owns_lock()` до присваивания. `other.mutex()==NULL`, `other.owns_lock()==false`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Объекты `std::unique_lock` не являются `CopyConstructible`, поэтому оператор копирующего присваивания отсутствует и имеется только этот оператор перемещающего присваивания.

Деструктор `std::unique_lock`

Уничтожает экземпляр `std::unique_lock` и разблокирует соответствующий мьютекс, если им владеет уничтоженный экземпляр.

Объявление

```
~unique_lock();
```

Результат

Если `this->owns_lock()` возвращает `true`, вызывает `this->mutex->unlock()`.

Выдает

Ничего не выдает.

Компонентная функция `std::unique_lock::swap`

Производит обмен принадлежности связанных с ними блокировок `unique_locks` выполнения между двумя объектами `std::unique_lock`.

Объявление

```
void swap(unique_lock& other) noexcept;
```

Результат

Если `other` владеет блокировкой мьютекса до вызова, то теперь этой блокировкой владеет `*this`. Если `*this` владеет блокировкой мьютекса до вызова, то теперь этой блокировкой владеет `other`.

Постусловия

Значение `this->mutex()` равно значению `other.mutex()` до вызова. Значение `other.mutex()` равно значению `this->mutex()` до вызова. Значение `this->owns_lock()` равно значению `other.owns_lock()` до вызова. Значение `other.owns_lock()` равно значению `this->owns_lock()` до вызова.

Выдает

Ничего не выдает.

Некомпонентная функция обмена для `std::unique_lock`

Обменивает владение ассоциированными блокировками мьютекса между двумя объектами `std::unique_lock`.

Объявление

```
void swap(unique_lock& lhs, unique_lock& rhs) noexcept;
```

Результат

```
lhs.swap(rhs)
```

Выдает

Ничего не выдает.

Компонентная функция `std::unique_lock::lock`

Получает блокировку мьютекса, связанную с `*this`.

Объявление

```
void lock();
```

Предусловия

```
this->mutex()!=NULL, this->owns_lock()==false
```

Результат

Вызывает `this->mutex()->lock()`.

Выдает

Любые исключения, выданные вызовом `this->mutex()->lock()`. Исключение `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если `this->mutex()==NULL`. Исключение `std::system_error` с кодом ошибки `std::errc::resource_deadlock_would_occur`, если на входе `this->owns_lock()==true`.

Постусловия

```
this->owns_lock()==true
```

Компонентная функция `std::unique_lock::try_lock`

Пытается получить блокировку мьютекса, связанную с `*this`.

Объявление

```
bool try_lock();
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::unique_lock`, должен быть `Lockable`.

```
this->mutex()!=NULL, this->owns_lock()==false
```

Результат

Вызывает `this->mutex()->try_lock()`.

Возвращает

`true`, если вызов `this->mutex()->try_lock()` возвращает `true`, или `false` в противном случае.

Выдает

Любые исключения, выданные вызовом `this->mutex()->try_lock()`. Исключение `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если `this->mutex()==NULL`. Исключение `std::system_error` с кодом ошибки `std::errc::resource_deadlock_would_occur`, если на входе `this->owns_lock()==true`.

Постусловия

Если функция возвращает `true`, `this->owns_lock()==true`, в противном случае `this->owns_lock()==false`.

Компонентная функция `std::unique_lock::unlock`

Освобождает блокировку мьютекса, связанную с `*this`.

Объявление

```
void unlock();
```

Предусловия

```
this->mutex()!=NULL, this->owns_lock()==true
```

Результат

Вызывает `this->mutex()->unlock()`.

Выдает

Любые исключения, выданные вызовом `this->mutex()->try_unlock()`. Исключения `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если на входе `this->owns_lock()==false`.

Постусловия

```
this->owns_lock()==false
```

Компонентная функция `std::unique_lock::try_lock_for`

Пытается получить блокировку мьютекса, связанную с `*this`, в течение заданного времени.

Объявление

```
template<typename Rep, typename Period>
bool try_lock_for(
    std::chrono::duration<Rep, Period> const& relative_time);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::unique_lock`, должен быть `TimedLockable`.

```
this->mutex()!=NULL, this->owns_lock()==false
```

Результат

Вызывает `this->mutex()->try_lock_for(relative_time)`.

Возвращает

`true`, если вызов `this->mutex()->try_lock_for()` возвращает `true`, или `false` в противном случае.

Выдает

Любые исключения, выданные вызовом `this->mutex()->try_lock_for()`. Исключение `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если `this->mutex()==NULL`. Исключение `std::system_error` с кодом ошибки `std::errc::resource_deadlock_would_occur`, если на входе `this->owns_lock()==true`.

Постусловия

Если функция возвращает `true`, `this->owns_lock()==true`, в противном случае `this->owns_lock()==false`.

Компонентная функция `std::unique_lock::try_lock_until`

Пытается получить блокировку мьютекса, связанную с `*this`, в течение заданного времени.

Объявление

```
template<typename Clock, typename Duration>
bool try_lock_until(
    std::chrono::time_point<Clock, Duration> const& absolute_time);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::unique_lock`, должен быть `TimedLockable`.

```
this->mutex()!=NULL, this->owns_lock()==false
```

Результат

Вызывает `this->mutex()->try_lock_until(absolute_time)`.

Возвращает

`true`, если вызов `this->mutex()->try_lock_until()` возвращает `true`, или `false` в противном случае.

Выдает

Любые исключения, выданные вызовом `this->mutex()->try_lock_until()`. Исключение `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если `this->mutex()==NULL`. Исключение `std::system_error` с кодом ошибки `std::errc::resource_deadlock_would_occur`, если на входе `this->owns_lock()==true`.

Постусловие

Если функция возвращает `true`, `this->owns_lock()==true`, в противном случае `this->owns_lock()==false`.

Булева компонентная функция `std::unique_lock::operator`

Проверяет принадлежность блокировки мьютекса объекту `*this`.

Объявление

```
explicit operator bool() const noexcept;
```

Возвращает

`this->owns_lock()`

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Это оператор явного преобразования, поэтому он подразумевается вызывается в контексте, где результат используется как булево значение, а не там, где результат будет рассматриваться как целочисленное значение, равное 0 или 1.

Компонентная функция `std::unique_lock::owns_lock`

Проверяет принадлежность блокировки мьютекса объекту `*this`.

Объявление

```
bool owns_lock() const noexcept;
```

Возвращает

`true`, если `*this` владеет блокировкой мьютекса, или `false` в противном случае.

Выдает

Ничего не выдает.

Компонентная функция `std::unique_lock::mutex`

Возвращает мьютекс, связанный с `*this`, если таковой имеется.

Объявление

```
mutex_type* mutex() const noexcept;
```

Возвращает

Указатель на мьютекс, связанный с `*this`, если таковой имеется, или `NULL` в противном случае.

Выдает

Ничего не выдает.

Компонентная функция `std::unique_lock::release`

Возвращает мьютекс, связанный с `*this`, если таковой имеется, и освобождает эту связь.

Объявление

```
mutex_type* release() noexcept;
```

Результат

Разрывает связь мьютекса с `*this` без разблокировки любых удерживаемых блокировок.

Возвращает

Указатель на мьютекс, связанный с `*this` до вызова, если таковой имелся, или `NULL` в противном случае.

Постусловия

```
this->mutex()==NULL, this->owns_lock()==false
```

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Если до вызова `this->owns_lock()` возвратил бы `true`, то тогда ответственность за разблокировку мьютекса возлагалась бы на вызывающий код.

Г.5.10. Шаблон класса `std::shared_lock`

Шаблон класса `std::shared_lock` предоставляет эквивалент `std::unique_lock`, за исключением того, что он получает совместно используемую, а не исключительную блокировку. Тип блокируемого мьютекса задается параметром шаблона `Mutex`, который должен обладать свойством `SharedLockable`. Как правило, указанный мьютекс блокируется в конструкторе и разблокируется в деструкторе, но предоставляются также дополнительные конструкторы и компонентные функции, допускающие

другие возможности. Тем самым предоставляется средство блокировки мьютекса для блока кода и предоставления гарантии, что мьютекс будет разблокирован при выходе управления из блока независимо от того, произошел он по достижении конца блока за счет использования инструкции управляющего потока, например `break` или `return`, или за счет выдачи исключения. Все экземпляры `std::shared_lock` подходят для использования с параметром `lockable` для функций ожидания `std::condition_variable_any`.

Каждый объект `std::shared_lock<Mutex>` является `lockable`. Если в дополнение к этому предоставляемый тип `Mutex` является `SharedTimedLockable`, то `std::shared_lock<Mutex>` также является `TimedLockable`.

Экземпляры `std::shared_lock` являются `MoveConstructible` и `MoveAssignable`, но не `CopyConstructible` или `CopyAssignable`.

Определение класса

```
template <class Mutex>
class shared_lock
{
public:
    typedef Mutex mutex_type;

    shared_lock() noexcept;
    explicit shared_lock(mutex_type& m);
    shared_lock(mutex_type& m, adopt_lock_t);
    shared_lock(mutex_type& m, defer_lock_t) noexcept;
    shared_lock(mutex_type& m, try_to_lock_t);

    template<typename Clock,typename Duration>
    shared_lock(
        mutex_type& m,
        std::chrono::time_point<Clock,Duration> const& absolute_time);

    template<typename Rep,typename Period>
    shared_lock(
        mutex_type& m,
        std::chrono::duration<Rep,Period> const& relative_time);

    ~shared_lock();

    shared_lock(shared_lock const& ) = delete;
    shared_lock& operator=(shared_lock const& ) = delete;

    shared_lock(shared_lock&& );
    shared_lock& operator=(shared_lock&& );

    void swap(shared_lock& other) noexcept;

    void lock();
    bool try_lock();
    template<typename Rep, typename Period>
    bool try_lock_for(
        std::chrono::duration<Rep,Period> const& relative_time);
```

```

template<typename Clock, typename Duration>
bool try_lock_until(
    std::chrono::time_point<Clock,Duration> const& absolute_time);
void unlock();

explicit operator bool() const noexcept;
bool owns_lock() const noexcept;
Mutex* mutex() const noexcept;
Mutex* release() noexcept;
};

```

Конструктор по умолчанию std::shared_lock

Создает экземпляр `std::shared_lock` без связанного с ним мьютекса.

Объявление

```
shared_lock() noexcept;
```

Результат

Создает экземпляр `std::shared_lock`, не имеющий связанного с ним мьютекса.

Постусловия

```
this->mutex()==NULL, this->owns_lock()==false
```

Блокирующий конструктор std::shared_lock

Создает экземпляр `std::shared_lock`, получающий совместно используемую блокировку предоставленного мьютекса.

Объявление

```
explicit shared_lock(mutex_type& m);
```

Результат

Создает экземпляр `std::shared_lock`, ссылающийся на предоставленный мьютекс. Вызывает `m.lock_shared()`.

Выдает

Любые исключения, выданные `m.lock_shared()`.

Постусловия

```
this->owns_lock()==true, this->mutex()==&m
```

Принимающий блокировку конструктор std::shared_lock

Создает экземпляр `std::shared_lock`, владеющий блокировкой предоставленного мьютекса.

Объявление

```
shared_lock(mutex_type& m, std::adopt_lock_t);
```

Предусловия

Вызывающий поток должен владеть совместно используемой блокировкой `m`.

Результат

Создает экземпляр `std::shared_lock`, ссылающийся на предоставленный мьютекс и получающий владение совместно используемой блокировкой `m`, удерживаемой вызывающим потоком.

Выдает

Ничего не выдает.

Постусловия

```
this->owns_lock()==true, this->mutex()==&m
```

Конструктор отложенной блокировки `std::shared_lock`

Создает экземпляр `std::shared_lock`, не владеющий блокировкой предоставленного мьютекса.

Объявление

```
shared_lock(mutex_type& m, std::defer_lock_t) noexcept;
```

Результат

Создает экземпляр `std::shared_lock`, ссылающийся на предоставленный мьютекс.

Выдает

Ничего не выдает.

Постусловия

```
this->owns_lock()==false, this->mutex()==&m
```

Конструктор попытки блокирования `std::shared_lock`

Создает экземпляр `std::shared_lock`, связанный с предоставленным мьютексом и предпринимающий попытку получения совместно используемой блокировки этого мьютекса.

Объявление

```
shared_lock(mutex_type& m, std::try_to_lock_t);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::shared_lock`, должен быть `Lockable`.

Результат

Создает экземпляр `std::shared_lock`, ссылающийся на предоставленный мьютекс. Вызывает `m.try_lock_shared()`.

Выдает

Ничего не выдает.

Постусловия

`this->owns_lock()` возвращает результат вызова `m.try_lock_shared()`, `this->mutex()==&m`.

Конструктор попытки блокирования с тайм-аутом продолжительностью `std::shared_lock`

Создает экземпляр `std::shared_lock`, связанный с предоставленным мьютексом и предпринимающий попытку получения совместно используемой блокировки этого мьютекса.

Объявление

```
template<typename Rep,typename Period>
shared_lock(
    mutex_type& m,
    std::chrono::duration<Rep,Period> const& relative_time);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::shared_lock`, должен быть `SharedTimedLockable`.

Результат

Создает экземпляр `std::shared_lock`, ссылающийся на предоставленный мьютекс. Вызывает `m.try_lock_shared_for(relative_time)`.

Выдает

Ничего не выдает.

Постусловия

`this->owns_lock()` возвращает результат вызова `m.try_lock_shared()`, `this->mutex()==&m`.

Конструктор попытки блокирования с тайм-аутом по моменту времени `time_point` `std::shared_lock`

Создает экземпляр `std::shared_lock`, связанный с предоставленным мьютексом и предпринимающий попытку получения совместно используемой блокировки этого мьютекса.

Объявление

```
template<typename Clock,typename Duration>
shared_lock(
    mutex_type& m,
    std::chrono::time_point<Clock,Duration> const& absolute_time);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::shared_lock`, должен быть `SharedTimedLockable`.

Результат

Создает экземпляр `std::shared_lock`, ссылающийся на предоставленный мьютекс. Вызывает `m.try_lock_shared_until(absolute_time)`.

Выдает

Ничего не выдает.

Постусловия

`this->owns_lock()` возвращает результат вызова `m.try_lock_shared()`, `this->mutex()==&m`.

Перемещающий конструктор `std::shared_lock`

Перемещает принадлежность совместно используемой блокировки из одного объекта `std::shared_lock` в только что созданный объект `std::shared_lock`.

Объявление

```
shared_lock(shared_lock&& other) noexcept;
```

Результат

Создает экземпляр `std::shared_lock`. Если `other` владел совместно используемой блокировкой мьютекса до вызова конструктора, то теперь этой блокировкой владеет только что созданный объект `std::shared_lock`.

Постусловия

Для только что созданного объекта `std::shared_lock`, `x`, результат вычисления `x.mutex()` равен значению `other.mutex()` до вызова конструктора, а результат вычисления `x.owns_lock()` равен значению `other.owns_lock()` до вызова конструктора. `other.mutex()==NULL`, `other.owns_lock()==false`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Объект `std::shared_lock` не является `CopyConstructible`, поэтому копирующий конструктор отсутствует и имеется только этот перемещающий конструктор.

Перемещающий оператор присваивания `std::shared_lock`

Перемещает принадлежность совместно используемой блокировки из одного объекта `std::shared_lock` в другой объект `std::shared_lock`.

Объявление

```
shared_lock& operator=(shared_lock&& other) noexcept;
```

Результат

Если `this->owns_lock()` возвращает `true` до вызова, вызывает `this->unlock()`. Если `other` владел совместно используемой блокировкой мьютекса до присваивания, то теперь этой блокировкой владеет `*this`.

Постусловия

Значение `this->mutex()` равно значению `other.mutex()` до присваивания, а значение `this->owns_lock()` равно значению `other.owns_lock()` до присваивания. `other.mutex()==NULL`, `other.owns_lock()==false`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Объект `std::shared_lock` не является `CopyConstructible`, поэтому оператор копирующего присваивания отсутствует и имеется только этот оператор перемещающего присваивания.

Деструктор `std::shared_lock`

Уничтожает экземпляр `std::shared_lock` и разблокирует соответствующий мьютекс, если им владеет уничтоженный экземпляр.

Объявление

```
~shared_lock();
```

Результат

Если `this->owns_lock()` возвращает `true`, вызывает `this->mutex->unlock_shared()`.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_lock::swap`

Производит обмен принадлежностью связанных с ними блокировок `shared_locks` выполнения между двумя объектами `std::shared_locks`.

Объявление

```
void swap(shared_lock& other) noexcept;
```

Результат

Если `other` владеет блокировкой мьютекса до вызова, то теперь этой блокировкой владеет `*this`. Если `*this` владеет блокировкой мьютекса до вызова, то теперь этой блокировкой владеет `other`.

Постусловия

Значение `this->mutex()` равно значению `other.mutex()` до вызова. Значение `other.mutex()` равно значению `this->mutex()` до вызова. Значение `this->owns_lock()` равно значению `other.owns_lock()` до вызова. Значение `other.owns_lock()` равно значению `this->owns_lock()` до вызова.

Выдает

Ничего не выдает.

Некомпонентная функция обмена для `std::shared_lock`

Производит обмен принадлежности связанных с ними блокировок мьютекса между двумя объектами `std::shared_lock`.

Объявление

```
void swap(shared_lock& lhs, shared_lock& rhs) noexcept;
```

Результат

```
lhs.swap(rhs)
```

Выдает

Ничего не выдает.

Компонентная функция `std::shared_lock::lock`

Получает совместно используемую блокировку мьютекса, связанную с `*this`.

Объявление

```
void lock();
```

Предусловия

```
this->mutex()!=NULL, this->owns_lock()==false
```

Результат

Вызывает `this->mutex()->lock_shared()`.

Выдает

Любые исключения, выданные вызовом `this->mutex()->lock_shared()`. Исключение `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если `this->mutex()==NULL`. Исключение `std::system_error` с кодом ошибки `std::errc::resource_deadlock_would_occur`, если на входе `this->owns_lock()==true`.

Постусловия

```
this->owns_lock()==true
```

Компонентная функция `std::shared_lock::try_lock`

Пытается получить совместно используемую блокировку мьютекса, связанную с `*this`.

Объявление

```
bool try_lock();
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::shared_lock`, должен быть `Lockable`.

```
this->mutex()!=NULL, this->owns_lock()==false
```

Результат

Вызывает `this->mutex()->try_lock_shared()`.

Возвращает

true, если вызов `this->mutex()->try_lock_shared()` возвращает true, или false в противном случае.

Выдает

Любые исключения, выданные вызовом `this->mutex()->try_lock_shared()`. Исключение `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если `this->mutex()==NULL`. Исключение `std::system_error` с кодом ошибки `std::errc::resource_deadlock_would_occur`, если на входе `this->owns_lock()==true`.

Постусловия

Если функция возвращает true, `this->owns_lock()==true`, в противном случае `this->owns_lock()==false`.

Компонентная функция `std::shared_lock::unlock`

Освобождает совместно используемую блокировку мьютекса, связанную с `*this`.

Объявление

```
void unlock();
```

Предусловия

```
this->mutex()!=NULL, this->owns_lock()==true
```

Результат

Вызывает `this->mutex()->unlock_shared()`.

Выдает

Любые исключения, выданные вызовом `this->mutex()->try_lock_shared()`. Исключение `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если на входе `this->owns_lock()==false`.

Постусловия

```
this->owns_lock()==false
```

Компонентная функция `std::shared_lock::try_lock_for`

Пытается получить совместно используемую блокировку мьютекса, связанную с `*this`, в течение заданного времени.

Объявление

```
template<typename Rep, typename Period>
bool try_lock_for(
    std::chrono::duration<Rep,Period> const& relative_time);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::shared_lock`, должен быть `SharedTimedLockable`.

```
this->mutex()!=NULL, this->owns_lock()==false
```

Результат

Вызывает `this->mutex()->try_lock_shared_for(relative_time)`.

Возвращает

`true`, если вызов `this->mutex()->try_lock_shared_for()` возвращает `true`, или `false` в противном случае.

Выдает

Любые исключения, выданные вызовом `this->mutex()->try_lock_shared_for()`. Исключение `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если `this->mutex()==NULL`. Исключение `std::system_error` с кодом ошибки `std::errc::resource_deadlock_would_occur`, если на входе `this->owns_lock()==true`.

Постусловия

Если функция возвращает `true`, `this->owns_lock()==true`, в противном случае `this->owns_lock()==false`.

Компонентная функция `std::shared_lock::try_lock_until`

Пытается получить совместно используемую блокировку мьютекса, связанную с `*this`, в течение заданного времени.

Объявление

```
template<typename Clock, typename Duration>
bool try_lock_until(
    std::chrono::time_point<Clock,Duration> const& absolute_time);
```

Предусловия

Тип `Mutex`, использованный для создания экземпляра `std::shared_lock`, должен быть `SharedTimedLockable`.

`this->mutex()!=NULL`, `this->owns_lock()==false`

Результат

Вызывает `this->mutex()->try_lock_shared_until(absolute_time)`.

Возвращает

`true`, если вызов `this->mutex()->try_lock_shared_until()` возвращает `true`, или `false` в противном случае.

Выдает

Любые исключения, выданные вызовом `this->mutex()->try_lock_shared_until()`. Исключение `std::system_error` с кодом ошибки `std::errc::operation_not_permitted`, если `this->mutex()==NULL`. Исключение `std::system_error` с кодом ошибки `std::errc::resource_deadlock_would_occur`, если на входе `this->owns_lock()==true`.

Постусловие

Если функция возвращает `true`, `this->owns_lock()==true`, в противном случае `this->owns_lock()==false`.

Булева компонентная функция `std::shared_lock::operator`

Проверяет принадлежность блокировки совместно используемого мьютекса объекту `*this`.

Объявление

```
explicit operator bool() const noexcept;
```

Возвращает

```
this->owns_lock()
```

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Это оператор явного преобразования, поэтому он подразумевается вызывается в контексте, где результат используется как булево значение, а не там, где результат будет рассматриваться как целочисленное значение, равное 0 или 1.

Компонентная функция `std::shared_lock::owns_lock`

Проверяет принадлежность блокировки совместно используемого мьютекса объекту `*this`.

Объявление

```
bool owns_lock() const noexcept;
```

Возвращает

`true`, если `*this` владеет совместно используемой блокировкой мьютекса, или `false` в противном случае.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_lock::mutex`

Возвращает мьютекс, связанный с `*this`, если таковой имеется.

Объявление

```
mutex_type* mutex() const noexcept;
```

Возвращает

Указатель на мьютекс, связанный с `*this`, если таковой имеется, или `NULL` в противном случае.

Выдает

Ничего не выдает.

Компонентная функция `std::shared_lock::release`

Возвращает мьютекс, связанный с `*this`, если таковой имеется, и освобождает эту связь.

Объявление

```
mutex_type* release() noexcept;
```

Результат

Разрывает связь мьютекса с `*this` без разблокировки любых удерживаемых блокировок.

Возвращает

Указатель на мьютекс, связанный с `*this` до вызова, если таковой имелся, или `NULL` в противном случае.

Постусловия

```
this->mutex()==NULL, this->owns_lock()==false
```

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Если до вызова `this->owns_lock()` возвратил бы `true`, то тогда ответственность за разблокировку мьютекса возлагалась бы на вызывающий код.

Г.5.11. Шаблон функции `std::lock`

Шаблон функции `std::lock` предоставляет средство блокировки более чем одного мьютекса одновременно, без риска возникновения взаимной блокировки, получаемой в результате несогласованного порядка блокировок.

Объявление

```
template<typename LockableType1, typename... LockableType2>  
void lock(LockableType1& m1, LockableType2& m2...);
```

Предусловия

Типы предоставленных блокируемых объектов, `LockableType1`, `LockableType2`, ..., должны быть `Lockable`.

Результат

Получает блокировку каждого из предоставленных блокируемых объектов, `m1`, `m2`, ..., путем неуказанной последовательности вызовов компонентных функций `lock()`, `try_lock()` и `unlock()` этих типов, избегающих взаимной блокировки.

Постусловия

Текущий поток владеет блокировкой каждого из предоставленных блокируемых объектов.

Выдает

Любые исключения, выданные вызовами `lock()`, `try_lock()` и `unlock()`.

ПРИМЕЧАНИЕ

Если исключение распространяется за пределы вызова `std::lock`, то функция `unlock()` должна была быть вызвана для любого из объектов `m1`, `m2`, ..., для которого блокировка была получена в функции путем вызова `lock()` или `try_lock()`.

Г.5.12. Шаблон функции `std::try_lock`

Шаблон функции `std::try_lock` позволяет предпринять попытку блокировки сразу набора блокируемых объектов, чтобы либо все они стали заблокированы, либо ни один из них не был заблокирован.

Объявление

```
template<typename LockableType1, typename... LockableType2>
int try_lock(LockableType1& m1, LockableType2& m2...);
```

Предусловия

Типы предоставленных блокируемых объектов, `LockableType1`, `LockableType2`, ..., должны быть `Lockable`.

Результат

Пытается получить блокировку каждого из предоставленных блокируемых объектов, `m1`, `m2`, ..., путем вызова `try_lock()` по очереди для каждого из них. Если вызов `try_lock()` привел к возвращению значения `false` или выдал исключение, уже полученная блокировка освобождается путем вызова функции `unlock()` в отношении соответствующего блокируемого объекта.

Возвращает

-1, если были получены все блокировки (каждый вызов `try_lock()` привел к возвращению значения `true`), в противном случае — начинающийся с нуля индекс объекта, для которого вызов `try_lock()` привел к возвращению значения `false`.

Постусловия

Если функция вернула -1, то текущий поток владеет блокировкой каждого из предоставленных блокируемых объектов. В противном случае все блокировки, полученные в результате ее вызова, освобождаются.

Выдает

Любые исключения, выданные при вызове функции `try_lock()`.

ПРИМЕЧАНИЕ

Если исключение распространяется за пределы вызова `std::try_lock`, то функция `unlock()` должна была быть вызвана для любого из объектов `m1`, `m2`, ..., для которого блокировка была получена в функции путем вызова `lock()` или `try_lock()`.

Г.5.13. Класс `std::once_flag`

Экземпляры класса `std::once_flag` используются вместе с шаблоном функции `std::call_once`, чтобы гарантировать, что конкретная функция вызывается только один раз, даже если вызов одновременно инициируется сразу несколькими потоками.

Экземпляры `std::once_flag` не являются `CopyConstructible`, `CopyAssignable`, `MoveConstructible` или `MoveAssignable`.

Определение класса

```
struct once_flag
{
    constexpr once_flag() noexcept;

    once_flag(once_flag const& ) = delete;
    once_flag& operator=(once_flag const& ) = delete;
};
```

Конструктор по умолчанию `std::once_flag`. Конструктор `std::once_flag` создает новый экземпляр `std::once_flag` в состоянии, показывающем, что связанная функция не была вызвана.

Объявление

```
constexpr once_flag() noexcept;
```

Результат

Создает новый экземпляр `std::once_flag` в состоянии, показывающем, что связанная функция не была вызвана. Поскольку конструктор помечен спецификатором `constexpr`, экземпляр со статической продолжительностью хранения создается в качестве составной части этапа статической инициализации, что исключает возможность возникновения гонок за данными и проблем, связанных с порядком инициализации.

Г.5.14. Шаблон функции `std::call_once`

Шаблон функции `std::call_once` используется с экземпляром `std::once_flag`, чтобы гарантировать, что конкретная функция вызывается только один раз, даже если вызов одновременно инициируется сразу несколькими потоками.

Объявление

```
template<typename Callable, typename... Args>
void call_once(std::once_flag& flag, Callable func, Args args...);
```

Предусловия

Выражение `INVOKE(func, args)` допустимо для предоставленных значений `func` и `args`. Объекты типа `Callable` и все компоненты `Args` являются `MoveConstructible`.

Результат

Вызовы `std::call_once` в отношении того же самого объекта `std::once_flag` выполняются последовательно.